

Shahjalal University of Science & Technology

Software Engineering

Institute of Information and Communication Technology

Course Code: SWE450



An Empirical Approach toward Identifying the Core Sentiments of Bengali Music Using Machine Learning and Neural Network Classifiers.

Submitted By

Mynul Islam

Reg. No.: 2018831059

Software Engineering

IICT, SUST

Md. Sabit Islam

Reg. No.: 2018831061

Software Engineering

IICT, SUST

Md Abu Salman

Reg. No.: 2018831067

Software Engineering

IICT, SUST

Supervisor

Dr. Ahsan Habib

Associate Professor

Institute of Information and Communication Technology

Shahjalal University of Science and Technology

February 10, 2024

An Empirical Approach toward Identifying the Core Sentiments of Bengali Music Using Machine Learning and Neural Network Classifiers.



A Thesis submitted to the Institute of Information and Communication Technology,
Shahjalal University of Science and Technology, in partial fulfillment of the
requirements for the degree of B.Sc.(Engg.) in Software Engineering.

Submitted By

Mynul Islam

Reg. No.: 2018831059

Software Engineering

IICT, SUST

Md Sabit Islam

Reg. No.: 2018831061

Software Engineering

IICT, SUST

Md Abu Salman

Reg. No.: 2018831067

Software Engineering

IICT, SUST

Supervisor

Dr. Ahsan Habib

Associate Professor

Institute of Information and Communication Technology

Shahjalal University of Science and Technology

February 10, 2024

Recommendation of the Thesis Supervisor

To Whom It May Concern

This letter is to certify that, the thesis entitled **An Empirical Approach toward Identifying the Core Sentiments of Bengali Music Using Machine Learning and Neural Network Classifiers.** undertaken by the students **Mynul Islam, Md. Sabit Islam Bhuiya and Md. Abu Salman Hossain** is under my supervision. I, hereby, agree that the thesis can be submitted for examination.

Dr. Ahsan Habib

Associate Professor

Institute of Information and Communication Technology

Shahjalal University of Science & Technology, Sylhet

Date: February 10, 2024

Approval of the Thesis

Students Name: Mynul Islam, Md. Sabit Islam Bhuiya, Md. Abu Salman Hossian

Thesis Title: An Empirical Approach toward Identifying the Core Sentiments of Bengali Music Using Machine Learning and Neural Network Classifiers.

This is to certify that the above mentioned thesis, submitted by the students named *above* in February, 2024 as part of the requirements of the course **SWE 450**, is being approved by the Software Engineering, Institute of Information and Communication Technology as a partial fulfillment of the B.Sc.(Engg.) degrees of the above students.

Director of IICT

Prof Dr. M. Jahirul Islam

PhD, PEng

Chairman,

Exam. Committee

Prof. Dr. M. Jahirul Islam

PhD, PEng

Supervisor

Dr. Ahsan Habib

Associate Professor

Acknowledgement

First of all, a myriad thanks to the most gracious and most merciful God, the creator of the heavens and the earth. Then, a warm salutation to our parents by whom we emerged in this beautiful world. We would like to thank our supervisor **Dr. Ahsan Habib**, Associate Professor, IICT, SUST. He shared the principle concept of this study with us and guided us throughout the study. We are really grateful to have worked under his greatly knowledgeable supervision and express our greatest gratitude to her for his guidance.

We take this opportunity to express my profound gratitude and deep regards to **Maraz Mia**, for his exemplary guidance, mentoring and constant encouragement.

Contents

1	Introduction	1
1.1	Objective	2
1.1.1	Novelty of the Research	2
2	Related Work	5
2.0.1	Progress on Bengali NLP	5
2.0.2	Music Emotions	5
2.0.3	Related Study	6
3	Handling Data	8
3.0.1	Dataset Preparation	8
3.0.2	Data Labeling	10
4	Data Analysis and Preprocessing	11
4.0.1	Data Cleanup	11
4.0.2	Data Distribution Analysis	11
5	Data Acquisition Platform	15
5.1	Model Training and Evaluation	15
5.1.1	Performance Measurements	26
5.2	Some Example :	36
6	Conclusion and Prospective Scope	40

List of Figures

3.1	Pseudo Code of the Verse separation method	10
4.1	Pseudo Code of the Verse separation method	12
4.2	Use of GloVe	13
4.3	Example of GloVe Vector Space	14
5.1	Bidirectional LSTM Network	19
5.2	Model Organization (CNN)	22
5.3	Model Organization (BLSTM)	23
5.4	Model Organization (BERT)	23
5.5	Model Parameters	24
5.6	BLSTM Training Parameters	24
5.7	Overall Process	26
5.8	Training and Testing Data Proportion	27
5.9	Resorted Batch Size	28
5.10	F1 Measure for Each Class	30
5.11	F1 Measure for Each Class	32
5.12	Accuracy BLTM	33
5.13	Loss CNN	34
5.14		36
5.15	Output of a sample Input	36
5.16		37
5.17	Output of a sample Input	37
5.18		37
5.19	Output of a sample Input	38
5.20		38
5.21	Output of a sample Input	38
5.22		39
5.23	Output of a sample Input	39

List of Tables

3.1	Dataset Details	8
3.2	Labels of our Dataset	10
5.1	Precision of CNN, BLSTM,BERT models	35
5.2	Recall of CNN, BLSTM,BERT models	35
5.3	F1 Score of CNN, BLSTM,BERT models	35

Chapter 1

Introduction

Identifying Emotions in Music Through Lyrics: A Powerful Tool for Personalized Experiences: Music has a profound impact on our lives, serving as a powerful emotional outlet and source of connection. Analyzing the sentiments and moods conveyed in lyrics, a process known as music emotion detection from textual data, opens exciting possibilities for a range of applications. Recent advancements in machine learning and neural networks have propelled music emotion detection into the forefront of Natural Language Processing (NLP) tasks. This allows us to automatically capture the emotional essence of music through its lyrics.

Understanding Sentiment: The Foundation of Emotion Analysis:

Sentiment refers to an individual's subjective opinion or perception of a topic, shaped by the underlying emotions it evokes. Sentiment analysis, a field concerned with deciphering emotions and opinions expressed by humans, has seen continuous progress over the past few decades.

Emotion extraction from textual data follows a structured approach, leveraging resources written in natural language [1]. **Why Bengali Music Deserves Attention:**

Music plays an inseparable role in human life, often serving as a form of emotional release for many. In the realm of music literacy, which encompasses poetry and lyrical works, Bengali boasts a wealth of world-class songwriters. However, much of this vast treasure trove remains undigitized. Despite the incomplete digitization, the popularity of Bengali music is witnessing a surge, with its captivating content transcending national boundaries and attracting listeners from diverse backgrounds. The rise of online communities and platforms like YouTube, Amazon, and Spotify has facilitated seamless content sharing and access, allowing individuals from various groups, ethnicities, and cultures to engage with the same music [1].

Lyrics: The Heart of the Song

Among the various elements of a song, such as vocals, instrumentation, genre, and rhythm, lyrics hold a unique significance. They directly impact human emotions and connect with listeners on a personal level. Often, the content of lyrics reflects a particular perspective on life and even the songwriter's personality. Music's ability to resonate with individuals at a personal level

allows them to choose and connect with songs that align with their current emotional state [2].

1.1 Objective

- **Multi-Class Classification Expansion:** Extend the multi-class classification approach to encompass a broader spectrum of emotions, including those specific to life and depression themes.
- **Advanced Model Training:**
Utilize advanced neural network models, such as BERT, BLSTM, for improved accuracy in emotion classification. Experiment with fine-tuning strategies to optimize model performance on the augmented dataset.
- **Feature Extraction Enhancement:**
Explore additional features and representations from both lyrics and audio data to create a more comprehensive hybrid mood classification system.
- **Real-Life Application:**
Investigate the implementation of the enhanced emotion analysis system in real-life digital systems, aiming for a more realistic and human-like experience in mood classification.

1.1.1 Novelty of the Research

Music emotion detection through textual analysis, powered by machine learning and neural networks, holds immense potential to revolutionize user experiences in music-related applications and even unlock new avenues for musical expression. Notably, our research goes beyond existing studies by introducing several innovative concepts and strategies, contributing significant novelty to the field. Let's explore these key advancements:

1.1.1.1 Verse Based Approach

Songs consist of many different components: lyrics, verses, refrain/chorus, and meter. Lyrics are a group of words that make up a song and are usually broken up into verses. Verses are a grouped portion of a song, which can also be found in poetry. The refrain of a song is the repeated part, usually broken up throughout the song. The meter is the measurement of regularly occurring beats or bars. Songs can have one single verse or many verses and refrains¹.

1.1.1.2 Custom Dataset Annotation

Annotating data is crucial for training machine learning models in Natural Language Processing (NLP). However, for Bengali, a "low resource language" [7], this task presented a unique

challenge. We had to create our own custom dataset with emotion labels, which proved quite intricate. The difficulty stemmed from the inherent richness and diversity of Bengali literary works. The vast spectrum of styles employed by writers like Fakir Lalon Shah, Nociketa, and countless others, coupled with the versatility of song content, made assigning precise emotion tags to individual verses a monumental undertaking.

1.1.1.3 Purpose and Outcome

Our research interest lies in the domain of Music Information Retrieval (MIR) and its application is dispersed in various aspects of life. The purpose of MIR is to develop computational methods to automatically analyze, organize, and retrieve music data. MIR involves the application of signal processing, machine learning, and information retrieval techniques to extract useful information from audio signals and music-related data, such as metadata, lyrics, and user-generated content. Our conducted model can be used in many different aspects of MIR such as:

Recommendation System

Music recommendation systems are computational tools designed to provide personalized music recommendations to users based on their music preferences, listening history, and other relevant data. These systems use various methods such as collaborative filtering, content-based filtering, and hybrid approaches to recommend music to users. By analyzing a listener's music taste and usual data, our proposed model then can automatically offer relevant music suggestions from user music history or from similarity networks.

Creation of New Music

Now, human aided tasks are being performed with computer aided systems and the power of automation has become overwhelming. This method can be resorted into music creation as it is capable of mimicking proximity to human nature by some means. Music creation can be defined as artificially created music with proper emotion on it. So, in this way our study can be prolific.

Demographic Usage

Music reflects one's inner imaginations and thoughts in a more abstract way. A person generally listens to that kind of music which represents his/her personality or it may be a medium of expressing inner feelings [3]. So, it is viable to recognize a persons' social behavior and men-

tal stage by scrutinizing the listener's music taste and likelihood. By undergoing this method, it is possible to identify potential criminals, suicidal or depraved minded people etc. and later they can be treated with apropos legitimate actions.

Chapter 2

Related Work

In this section we will discuss the background study of Bengali NLP and research related to mood, sentiment or emotion prediction for Bengali music from textual data.

2.0.1 Progress on Bengali NLP

Bengali, also known as Bangla, holds a significant place among Indo-Iranian languages. With over 250 million speakers, it ranks as the sixth most spoken language globally, serving as the native tongue for Bangladeshis and holding the second-language status in India. Despite its vast reach, its digital resources remain remarkably limited compared to well-documented languages like English [3]. This poses a significant challenge, compounded by Bengali's intricate grammatical structures and diverse linguistic nuances. Several studies have explored various aspects of Bengali language processing, including sentiment analysis of social media comments, speech recognition, Music Information Retrieval (MIR), and text classification. However, the relative scarcity of digital resources necessitates further efforts to bridge the gap and unlock the full potential of this vibrant language in the digital age

2.0.2 Music Emotions

Emotions can be defined as the thoughts on a particular subject by reasoning internal feelings. At least seven so-called primary emotions are represented by cultural facial expressions: sadness, joy, fear, wrath, surprise, disgust, and contempt. Without anything to feel compassionate, nostalgic, or triumphant about, how can we experience these emotions? It's true that music can evoke real emotions rather than merely fantasies, but it's also true that some emotional feelings are more likely to do this than others. What we have described is a generally recognized understanding of what emotions are and how they work, despite differences in opinion among researchers regarding the specifics. But why should music even remotely be associated with emotion? Although it is thought that music can convey emotion, in reality, people typically show emotion by altering their facial or vocal expressions, gestures, or behavior patterns

rather than by playing or creating music. Sometimes a seemingly happy music can be turned into lugubrious one if some changes in vocal tone occurred. So, along with lyrical components, a vocalist's effort on a song is also important to express the real emotion of the song. However, the major role of conveying a sentimental issue is played by the lyrical part of music [3]. Thus, to fully extract the real meaning of a song, we have to deep dive into the lyricist's writing style and the real nature of the lyrics itself.

2.0.3 Related Study

Due to the intricacy of music lyrics and the ephemeral nature of human emotions, sentiment analysis of music from lyrics is a difficult task in natural language processing (NLP). Yet, recent developments in NLP have made it possible to use sentiment analysis and emotion recognition techniques to music lyrics in order to extract the emotional content of a song. Machine learning techniques have already been used to complete a significant amount of work in the field of sentiment analysis. Analysis of the emotional content or sentiment of songs is a topic that has not been extensively studied for low resource languages. Although being a relatively old subject of study, the majority of the work has only been done for languages with abundant resources. In their publications, Patra et al. suggested techniques that are required to categorize music by moods even from music files uploaded in social networks, with a maximum F measure of 68%. Another study conducted on hindi songs [4], were able to achieve an average F1-Score of 38% for multi-class classification. However some work has been done in Bengali in the field of sentiment analysis.

To our knowledge, however, a few studies on the topic of automatically detecting sentiment or emotion from Bengali literature recommended the use of sense-based affect score algorithms for Bengali in order to track emotions depending on topic or event. But, most of the work mainly dealt in a polarity wise strategy, more specifically the songs were identified as either positive or negative (song with emotions such as happiness, motivational, excited, calm, astonished were considered as positive and angry, sad, anxious, gloomy, tensed, rebellious were considered negative) [6-8], [4], [5] where the accuracy is considerably higher because of binary classification (about 80-90%) and inconsideration for variety in songs collection. But, unfortunately we haven't found any profound work regarding multi-class emotion analysis for Bengali music yet so far. Before our work, a paper has published [6] on this topic and more relatable to our work and we were inspired by this and accuracy of 65% in multi-class classification and 80% in binary emotion classification.

Apart from this, some other study has been done on the related subject and peripheral to our study with multi-class classification method. A study on the emotion detection from youtube comments achieved an accuracy of 54% and another study was able to come up with a system with 80% accuracy on the music genre classification model [7]. Additionally, a more closely related research was able to gain an accuracy of 69% for the music lyrics classification [24]. As, our resorted method is based on verse and its perplexing characteristics made our proceeding much tougher thus, we got a low accuracy value.

Chapter 3

Handling Data

In natural language processing, a dataset is used to train a model to make accurate predictions or classifications. The quality and quantity of the data in the dataset can have a significant impact on the performance of the model. A dataset with insufficient or poor-quality data can lead to poor performance or even incorrect predictions.

3.0.1 Dataset Preparation

The primary hurdle in Bengali language processing for computers often lies in the absence of a standardized, annotated training dataset. To address this, we constructed a custom dataset comprised of song lyrics sung by diverse artists across various genres. Table I provides details on this dataset, which incorporates songs from 109 lyricists, meticulously compiled by the authors. It features over 10 songs per lyricist, totaling 38 unique contributors. Individual folders organize the lyrics based on the author, with each folder further containing multiple tracks from the same artist. The dataset encompasses a wide range of Bangladeshi music subgenres, including religious hymns, ethnic compositions, and contemporary pop songs. This diversity aims to capture the richness and complexity of Bengali musical expression.

While the original paper might have referenced seven basic emotions from some source [8], our study focused on five distinct emotions: happiness, love, sadness, depression, and ideal-

Objects	Count
Lyricists	109
Songs	325
Happy	123(Verse)
Love	279(Verse)
Sad	324(Verse)
Depression	262(Verse)
Life	249(Verse)
Total words	>10000

Table 3.1: Dataset Details

ism. This choice reflects the specific nuances we aimed to capture within the context of Bengali music lyrics. In some cases, verses conveyed emotions that didn't fall neatly into our designated categories. For such instances, we assigned the closest fitting label based on the overall emotional context of the verse. Figure I provides a visual representation of the data classes and their associated emotional contexts.

3.0.1.1 Verse Segmenting

Our approach to music emotion analysis differs from previous work primarily in how we handle individual verses. We recognize that different verses within a song can convey separate emotions, allowing a single song to encompass multiple emotional states. To address this, we implemented a "verse separation" method to identify and distinguish sections within song lyrics. This can be done manually or automatically, but we opted for the automated approach. However, our chosen dataset presented two distinct writing styles, requiring us to create a program specifically tailored to separate verses within it. Therefore, we employed two different sets of logic for verse extraction, which we will detail below:

3.0.1.2 Separated By New Line

It's actually the most frequent style resorted in the dataset. According to this style, verses are separated by a new line. It may contain the separator character (in most of the cases). But, the main splitter here is the extra new line after every verse.

3.0.1.3 Separated By Bengali Poetry Character

In this structure, the ending of a verse can be determined by a resting character frequently used in Bengali literature. The character comprises as '□'. Though it's not an universal rule of defining the end of the verse, in this dataset, along with the suggested style in 3.2.1, this pattern of writing also exists.

3.0.1.4 Separation Algorithm

We used a simple logic based verse separator algorithm and using the mother dataset, first we read the individual songs of every lyricist. A particular song has either of the writing styles stated at 3.0.1.1 so, first we need to identify which style category fits the underscored song. Then we separate the verses by the defined rules in 3.0.1.2 and 3.0.1.3 and finally write the newly found verses into our main dataset.

```

Song = Read StartOfFile to EndOfFile

If Lyrics_Type(Song) == Writing_Style_1:
    Verse = Separate_By_New_Line(Song)

Else:
    Verse = Separate_By_Resting_Character()

Write To Dataset(Verse)

```

Figure 3.1: Pseudo Code of the Verse separation method

Class	Lavel
Happy	1
Love	2
Sad	3
Depression	4
Life Oriented	5

Table 3.2: Labels of our Dataset

3.0.2 Data Labeling

In data labeling, we assign tags or labels (like "cat" or "happy") to raw data like text or audio. These labels help machine learning models learn to recognize similar items when they encounter them later. For our study focusing on five emotions, we labelled our dataset with three numerical codes. We carefully selected annotators with expertise and enthusiasm for music and related themes. To ensure data quality, we compared annotations from different experts and achieved a high 95% similarity, demonstrating agreement and reliability. Table II provides a breakdown of our labelling scheme.

Chapter 4

Data Analysis and Preprocessing

4.0.1 Data Cleanup

Transforming raw lyrics into a reliable dataset for analysis requires meticulous preparation. Firstly, each lyric was carefully labeled with an appropriate emotion tag, creating the foundation for our study. Following this, unnecessary characters and stop words - common words lacking significant meaning in our context - were meticulously removed to enhance the efficiency and accuracy of the processing. Finally, we implemented a rule-based Bengali stemmer, essentially standardizing words by reducing them to their root form. This multi-step process ensured the quality and consistency of our dataset, laying the groundwork for robust and reliable analysis.

4.0.2 Data Distribution Analysis

In data analysis, understanding the distribution of your dataset is crucial. This refers to how your data points are spread out across different categories or values of a specific variable or feature. Knowing the distribution holds several key benefits:

- **Revealing Data Nature:**

It provides insights into the fundamental characteristics of your data, such as whether it tends to be clustered around certain values or evenly spread out. This understanding can be critical for choosing appropriate analytical techniques.

- **Guiding Analysis Approaches:**

Different types of distributions necessitate different statistical methods. Identifying the distribution helps you ensure you're using the right tools to draw accurate conclusions from your data.

```

Refined_Bangla_Sentence = []

For Each Bangla_Word in Bangla_Sentence:

    Valid_Word = ''
    For Each character in Bangla_Word:

        If character in Valid_Bengali_Character_Set:
            Valid_Word.Add(character)

        Else:
            Valid_Word = ''

    If Valid_Word Is Not in Stop_Words_Set:
        Refined_Bangla_Sentence.Add(Valid_Word)

Return Stemmed(Refined_Bangla_Sentence)

```

Figure 4.1: Pseudo Code of the Verse separation method

- **Identifying Potential Biases:**

Uneven distributions can sometimes indicate biases or imbalances in your data collection process. Recognizing these patterns early on allows you to address them and achieve more reliable results

4.0.2.1 Overview of Our Dataset

Our dataset is primarily kept in text format in “Google Sheet” with 4 columns:

- Lyricist, name of writers of the song from various platforms as our dataset is a custom dataset.
- Verse, our main data frame for model training and extracted from various songs.
- Label, appropriate tag for each emotion class.

Other information about our dataset is given in Table 3.1:

4.0.2.2 Object Class Frequency

We follow several schemes while annotating our dataset with respected emotion label and here how we did it:

- **Happy:** When a song portrays multiple emotions, but the primary sentiment is happiness, the verse is considered a happy verse if it exudes a joyful, cheerful, or positive mood.

This can include references to contentment, celebration, or anything associated with uplifting vibes.

- **Love:** When a song depicts more than one sentiment, if the dominating connotation is related to love, the verse is referred to as a love verse if it has an overall sentiment of affection, amorous, relational, or amiable mood, or if something is mentioned with lovely vibes.
- **Sad:** When a song depicts more than one emotion, but the prevailing meaning is sad, the song is said to have a sad verse if the verse as a whole has a gloomy, depressing, or unhappy tone, or if something is mentioned with a sad vibe.
- **Depression:** In cases where a song encompasses various emotions but predominantly conveys a sense of depression, the verse is labeled as a depression verse if it carries a somber, despondent, or bleak tone. Depiction of sorrow, hopelessness, or melancholy themes contributes to the overall depressed atmosphere.
- **Life Oriented:** A verse is deemed life-oriented when it encapsulates a broader perspective on existence, emphasizing themes related to life experiences, growth, and resilience. Such verses may convey a sense of wisdom, reflection, or motivation, focusing on the journey of life and its various aspects.

4.0.2.3 Features Extraction

Predicting a text's sentiment is the aim of sentiment analysis. And feature extraction is one of the most important parts of it. There are many feature extraction models available for natural language processing (NLP) like Bag-of-Words (BoW), Term Frequency-Inverse Document Frequency (TF-IDF), FastText, Word to Vector, Global Vectors for Word Representation (GloVe), etc. GloVe is used here to extract important features for the model.

```
TEXT.build_vocab(  
    train_data,  
    max_size = MAX_VOCAB_SIZE,  
    vectors = "glove.6B.100d"  
)
```

Figure 4.2: Use of GloVe

Specifically, 'glove.6B.100d' vector indicates GloVe embeddings from the "glove.6B" dataset,

which contains 6 billion tokens, and each word is represented by a 100-dimensional vector [9]. Using GloVe to transform the words in a text into numerical vectors is a well-known technique for feature extraction in sentiment analysis. With GloVe, we may represent a text as a matrix of word vectors, with each row being a word in the text, and therefore capture the semantic links between words. A machine learning model can then be used to predict the sentiment of the text using the generated vector representation. GloVe is a popular option for feature extraction in natural language processing for a number of reasons. GloVe offers pre-trained word embeddings that can be used to initialize the word embeddings in a neural network for a variety of NLP tasks. These word embeddings were trained on large text corpora. It is intended to record the connections among words in a corpus. In order to capture the meaning of words as they are employed in the text, it learns the vector representations of words based on the co-occurrence statistics of words in a huge corpus. This can be helpful for projects like document classification or sentiment analysis. Again When evaluating different embedding models, high-quality embeddings, the availability of pre-trained models, and ease of use are frequently used as a benchmark.

```
# GloVe identifications example:
IF 'ঢাকা' = 'বাংলাদেশ': 'নয়া দিল্লী' = ?
GloVe answer: 'ভারত'
```

Snippet IV: GloVe Example

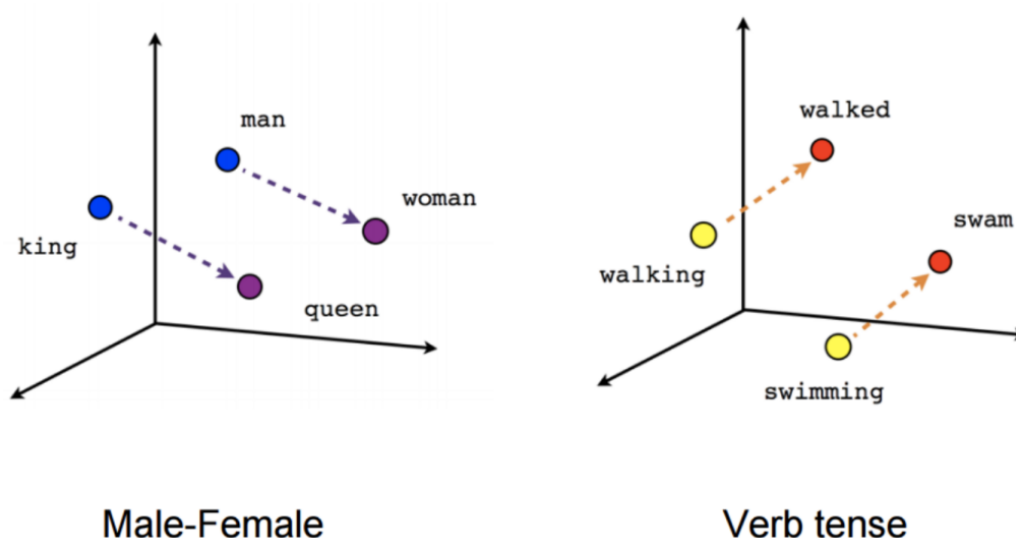


Figure 4.3: Example of GloVe Vector Space

Chapter 5

Data Acquisition Platform

5.1 Model Training and Evaluation

Several models have been used successfully for training sentiment analysis models. Recurrent neural networks (RNNs), convolutional neural networks (CNNs), and transformer models like BERT and GPT are a few of the more well-liked ones.

In the study train, Convolutional Neural Network (CNN) and Bidirectional Long Short Term Memory (BLSTM), Bidirectional Encoder Representations from Transformers (BERT) are initially presented and tested using the produced verse-base-Bengali-song dataset. Our goal is to test it on Transformer-based models such as GPT.

Here are several reasons why CNNs make an excellent choice for sentiment analysis.

CNNs are designed to automatically learn features from data, and can be especially effective for processing sequential data such as text. The convolutional layers in a CNN can extract different features at different levels of abstraction, which can be useful for sentiment analysis. CNNs are less prone to overfitting than other deep learning models such as RNNs. And faster training, interpretable results make it more popular. The features learned by the convolutional layers in a CNN can be visualized, which can help with understanding how the model is making predictions. The motivation for using verse-based data training at the initial stage is based on the research paper titled "Convolutional Neural Networks for Sentence Classification. [10]

Here are some advantages of the BLSTM model:

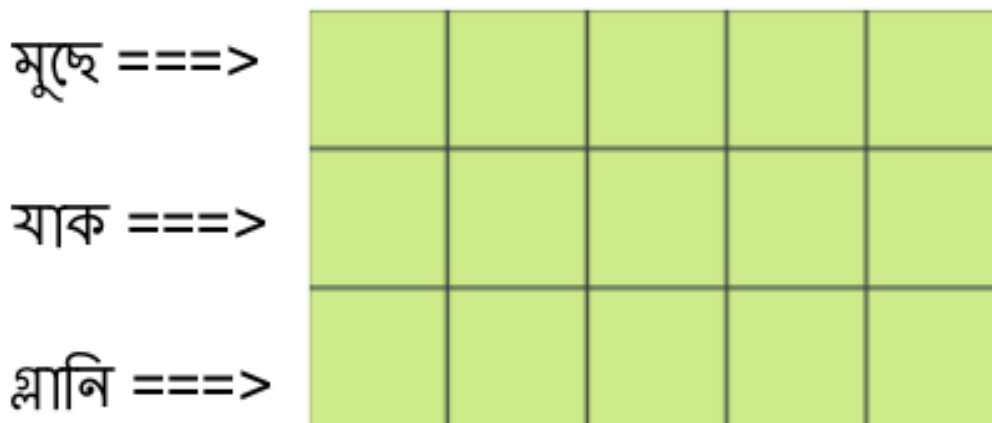
BLSTM models are specifically designed to capture long-term dependencies in sequential data, making them well-suited for analyzing text data where the order of words and phrases is important. Its bidirectional nature enables the model to simultaneously learn from both past and future contexts. This means that the model can capture more complex links between words and a word's meaning based on its circumstances. The models are highly flexible and can be adapted to different types of input data and different problem domains. And BLSTM models have been

shown to achieve state-of-the-art performance on a wide range of natural language processing tasks, including sentiment analysis. This means that they are a reliable and effective choice for sentiment analysis tasks, especially when working with complex and nuanced datasets. And we are motivated by the research of a paper entitled "Deep bidirectional LSTM network learning-based sentiment analysis for Arabic text. [11]

5.1.0.1 CNN Intuition

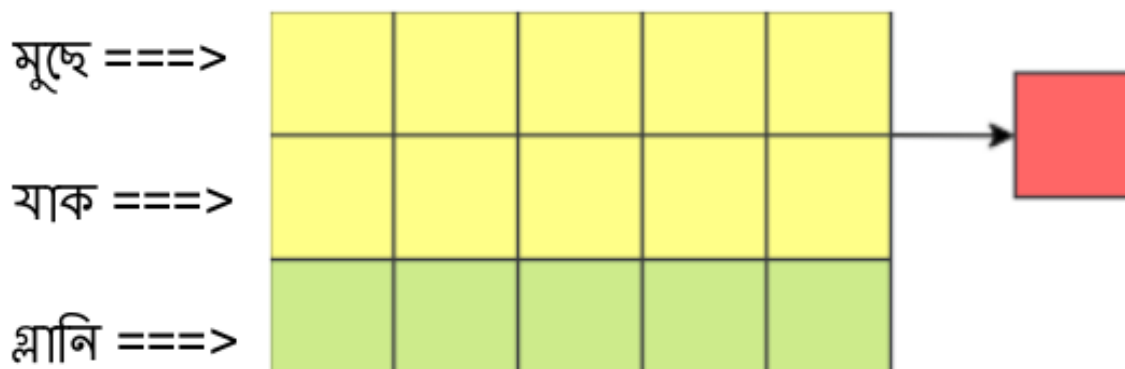
Traditionally, CNN consists of one or more convolutional layers, followed by one or more linear layers, and is used to analyze images. The convolutional layers employ filters, often referred to as kernels or receptive fields, to scan a picture and generate a modified version of the image. This altered image can be sent into either a linear or another convolutional layer. Each filter has a certain shape, such as a 3x3 filter, which covers an area of the image that is 3 pixels wide and 3 pixels high. Each element of the filter has a corresponding weight, so the 3x3 filter would have 9 weights. The intuitive concept behind learning the weights is that the convolutional layers act like feature extractors, extracting portions of the image that are most important for the CNN's goal, such as the presence of a nose, mouth, or a pair of eyes in the image if using a CNN to detect faces in an image. Hence why use CNNs on text? A 1x2 filter can examine two consecutive words in a piece of text, or a bi-gram, in a similar fashion to how a 3x3 filter can examine a patch of an image. It seems to make sense that the occurrence of specific bi-grams, tri-grams, and n-grams in the review will serve as a reliable indicator of the review's overall sentiment.

5.1.0.2 Understanding the CNN model

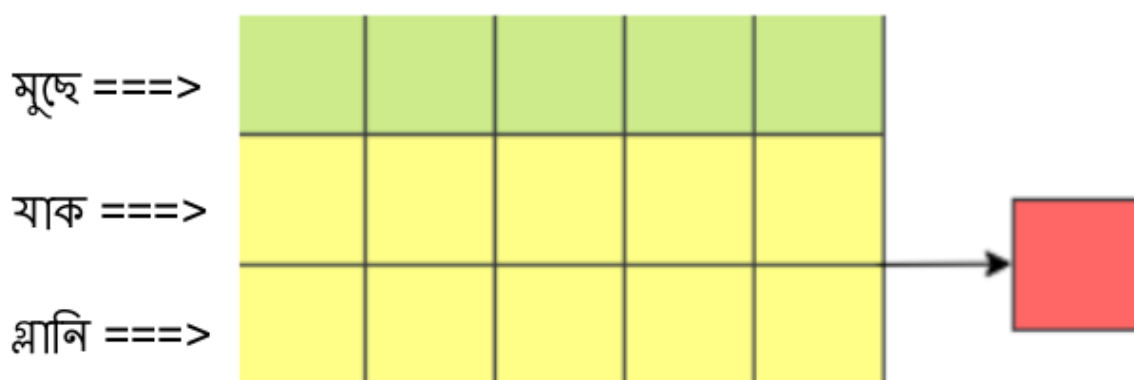


The first significant challenge is to visualize the use of CNNs for text. Text is one dimen-

sional while images are often two dimensional (for the time being ignoring the third "color" dimension). But, turning the words into word embeddings is the initial step. This allows us to see our words in two dimensions, with each word aligned along one axis and the components of vectors spanning the other. We can have a look at the above figure's two-dimensional representation of the contained message. A filter that is $n \times \text{emb dim}$ can then be used. Due to their width



being emb dim measurements, n consecutive words will be completely covered by this. Have a look at the image above, where the green word vectors are displayed. Here, we have a $[3 \times 5]$ "picture" tensor made up of 3 words with 5 dimensional embeddings. A $[2 \times 5]$ filter, depicted in yellow, will cover bi-grams, or two words at a time, and each element of the filter will have a weight assigned to it. This filter's output, which is shown in red, will be a single real number that represents the weighted sum of all the components it filters.



After covering the next bi-gram, the filter advances "down" the image (or across the sentence) to provide another output (weighted sum).

Nevertheless, the average pool function forecasts the ultimate value in this filtering process by using the highest value over a dimension rather than the average over a dimension.

5.1.0.3 Understanding the BLSTM Model

The BLSTM model is a type of Recurrent Neural Network (RNN) that consists of two LSTM layers - one processing the input sequence in a forward direction and the other processing it in a backward direction. Each LSTM layer consists of a memory cell and three gates (input, output, and forget gates) that control the flow of information through the cell. The output of each LSTM layer is the concatenation of its forward and backward outputs, which are passed onto the next layer or the output layer. **Input Encoding:** The input sequence is initially represented as a series of learnt representations of each word in the input sequence known as embeddings. Usually, these embeddings have already been trained on sizable datasets or have been trained alongside the model on the task-specific training set. The input sequence is denoted as $X = x_1, x_2, \dots, x_n$, where each x_i is an input feature vector at time i . **Forward Pass:** The first LSTM layer receives the input embeddings in a forward manner. The LSTM layer processes the input embedding at each time step and produces a hidden state as well as a cell state. These states convey to the following time step the context of the input sequence up to that point. **Backward Pass:** The second LSTM layer receives the input embeddings in reverse order. The LSTM layer processes the input embedding at each time step and produces a hidden state as well as a cell state. These states transmit to the preceding time step the context of the input sequence up to that point. **Output Generation:** Each LSTM layer's output is the result of concatenating its forward and backward outputs. The concatenated outputs are sent through one or more fully connected layers and a softmax activation function to create the BLSTM model's final output. The label with the highest probability is chosen as the anticipated sentiment by the softmax function, which creates a probability distribution across all potential sentiment labels.

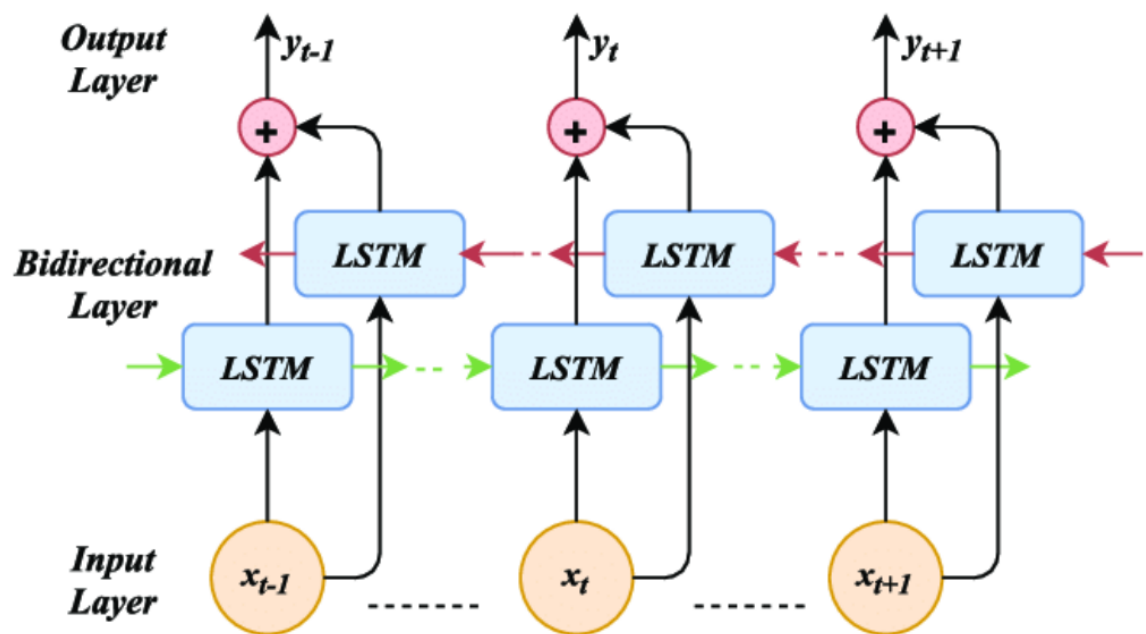


Figure 5.1: Bidirectional LSTM Network

Each LSTM layer consists of a memory cell C_t and three gates: the input gate i_t , the output gate o_t , and the forget gate f_t . The gates control the flow of information through the cell and are computed as follows:

$$i_t = \sigma(W_{ix}x_t + U_{ih}h_{(t-1)} + b_i)$$

$$o_t = \sigma(W_{ox}x_t + U_{oh}h_{(t-1)} + b_o)$$

$$f_t = \sigma(W_{fx}x_t + U_{fh}h_{(t-1)} + b_f)$$

where σ is the sigmoid activation function, W and U are weight matrices, b is the bias vector, and $h(t-1)$ is the output of the previous time step.

The cell state C_t and the hidden state h_t of the LSTM layer are updated as follows:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_{cx} + U_{ch_{(t-1)}} + b_c)$$

$$h_t = o_t \odot \tanh(C_t)$$

where $\odot$ denotes element-wise multiplication, $\tanh$ is the hyperbolic tangent activation function, and W_c , U_c , and b_c are weight matrices and bias vector for the cell state.

The output of each LSTM layer is the concatenation of its forward and backward outputs, which are denoted as $h(t)$ and $h'(t)$, respectively:

$$h(t) = h_t$$

$$h'(t) = h'_t$$

The final output of the BLSTM model is generated by passing the concatenated outputs through one or more fully connected layers and a softmax activation function. The softmax function generates a probability distribution over the possible sentiment labels, and the label with the highest probability is selected as the predicted sentiment.

5.1.0.4 Understanding BERT Model

Understanding the BERT (Bidirectional Encoder Representations from Transformers) model involves grasping its architecture, pre-training process, and its applications. Here's a simplified explanation:

- **Transformer Architecture:** BERT is built on the Transformer architecture, which was

introduced in the paper "Attention is All You Need." Transformers use attention mechanisms to process input data in parallel, allowing them to capture dependencies between words in a sentence more effectively.

- **Bidirectional Pre-training:** BERT introduces bidirectional context to pre-training. Traditional models, like those based on LSTMs or GRUs, processed text in one direction (either left-to-right or right-to-left). BERT, on the other hand, considers both directions simultaneously during pre-training. This bidirectional approach enables the model to capture context and relationships between words more comprehensively.
 - **Pre-training Objective:** BERT is pre-trained on large corpora of text data using two main tasks: Masked Language Model (MLM) and Next Sentence Prediction (NSP). MLM involves randomly masking some words in a sentence and training the model to predict those masked words based on the surrounding context. NSP involves determining if a pair of sentences appears consecutively in the original text or if they are randomly sampled. The model learns to predict whether the second sentence follows the first.
 - **Embeddings:** BERT generates contextualized word embeddings. Each word is embedded based on its context within a sentence, and these embeddings are dynamic, changing based on the context of the entire sentence.
 - **Fine-tuning:** After pre-training, BERT can be fine-tuned for specific downstream tasks. This involves training the model on a smaller dataset related to the target task, adjusting the weights of the pre-trained BERT model to perform well on that specific task.
 - **Applications:** BERT has achieved state-of-the-art results in a wide range of NLP tasks, such as text classification, named entity recognition, sentiment analysis, and question answering. Its ability to capture nuanced contextual relationships between words makes it particularly effective in understanding the meaning of language.
- In summary, BERT is a powerful NLP model that leverages bidirectional context and transformer architecture to pre-train on vast amounts of text data, enabling it to understand and represent the nuances of language, making it highly effective for various downstream tasks.

5.1.0.5 Implementation Details (CNN)

With `nn.Conv2d`, the convolutional layers are implemented. The number of "channels" in the image entering the convolutional layer is indicated by the `in_channels` argument. When employing text, however, only one channel is counted, the text itself, although in actual photos this is typically three (one channel for each of the red, blue, and green channels). The `out_channels` indicates how many filters there are, and the `kernel_size` indicates how big each filter is. As `n` is the size of the `n`-grams, each of our kernel sizes will be `[n x emb-dim]`. The ReLU activation function is used after the convolutional layers, and the tensors are then sent via the convolutional and pooling layers. The pooling layers' ability to handle phrases of various lengths is another appealing feature. Different batches contain sentences of various lengths, and the amount of the input to the convolutional layer determines the size of the output. Without the max pooling layer, the size of the input would determine the input to the linear layer (not what is desired). One option to rectify this would be to trim/pad all sentences to the same length, however with the max pooling layer we always know the input to the linear layer will be the total number of filters. Point to be noted is that there is an exception to this if the sentence(s) are shorter than the largest filter used. In this case, the sentences need to be padded to the length of the largest filter. Finally, dropout is performed on the concatenated filter outputs and then passes them through a linear layer to make the predictions.

```
import torch.nn as nn
import torch.nn.functional as F
class CNN(nn.Module):
    def __init__(self, vocab_size, embedding_dim, n_filters, filter_sizes, output_dim,
                 dropout, pad_idx):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.convs = nn.ModuleList([
            nn.Conv2d(in_channels = 1,
                      out_channels = n_filters,
                      kernel_size = (fs, embedding_dim))
            for fs in filter_sizes
        ])
        self.fc = nn.Linear(len(filter_sizes) * n_filters, output_dim)
        self.dropout = nn.Dropout(dropout)
    def forward(self, text):
        #text = [sent len, batch size]
        text = text.permute(1, 0)
        #text = [batch size, sent len]
        embedded = self.embedding(text)
        #embedded = [batch size, sent len, emb dim]
        embedded = embedded.unsqueeze(1)
        #embedded = [batch size, 1, sent len, emb dim]
        convd = [F.relu(conv(embedded)).squeeze(3) for conv in self.convs]
        #conv_n = [batch size, n_filters, sent len - filter_sizes[n]]
        pooled = [F.max_pool1d(conv, conv.shape[2]).squeeze(2) for conv in convd]
        #pooled_n = [batch size, n_filters]
        cat = self.dropout(torch.cat(pooled, dim = 1))
        #cat = [batch size, n_filters * len(filter_sizes)]
        return self.fc(cat)
```

Figure 5.2: Model Organization (CNN)

5.1.0.6 Implementation Details (BLSTM)

```
# Build BLSTM model
model = Sequential()
model.add(Embedding(MAX_NB_WORDS, EMBEDDING_DIM, input_length=X.shape[1]))
model.add(SpatialDropout1D(0.2))
model.add(Bidirectional(LSTM(100, dropout=0.2, recurrent_dropout=0.2, return_sequences=True)))
model.add(Bidirectional(LSTM(100, dropout=0.2, recurrent_dropout=0.2, return_sequences=True)))
model.add(Bidirectional(LSTM(100, dropout=0.2, recurrent_dropout=0.2)))
model.add(Dense(5, activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

Figure 5.3: Model Organization (BLSTM)

5.1.0.7 Implementation Details BERT

```
# from simpletransformers.classification import ClassificationModel
w1 = total_entry/(class_cnt*happy_cnt)
w2 = total_entry/(class_cnt*life_cnt)
w3 = total_entry/(class_cnt*sad_cnt)
w4=total_entry/(class_cnt*love_cnt)
w5=total_entry/(class_cnt*depression_cnt)
class_weight = [w1,w2,w3,w4,w5]
class_weight

bert2 = ClassificationModel('bert', 'sagorsarker/bangla-bert-base', num_labels=class_cnt, use_cuda=False,
                           weight=class_weight,
                           args={'reprocess_input_data': True,
                                'use_cached_eval_features': False,
                                'overwrite_output_dir': True,
                                'num_train_epochs': 3,
                                'silent': True})

bert2.train_model(train)
result, model_outputs, wrong_predictions = bert2.eval_model(train_eval)
bads = {}
for pred in wrong_predictions:
    if pred.label in bads:
        bads[pred.label] += 1
    else:
        bads[pred.label] = 1
print("wrong predictions:")
print(str(len(wrong_predictions)) + ' wrong out of ' + str(len(train_eval)))
print('Validation Accuracy : ', (len(train_eval)-len(wrong_predictions))/len(train_eval))
bads
```

Figure 5.4: Model Organization (BERT)

5.1.0.8 Model Instance Creation (CNN)

Here in the code snippet, CNN model instance is created with all the important parameters. Dropout indicates the regularization parameter to control the overfit problem in terms of test data

```

INPUT_DIM = len(TEXT.vocab)
EMBEDDING_DIM = 100
N_FILTERS = 100
FILTER_SIZES = [1,2,3,4,5]
OUTPUT_DIM = len(LABEL.vocab)
DROPOUT = 0.2
PAD_IDX = TEXT.vocab.stoi[TEXT.pad_token]

model = CNN(INPUT_DIM, EMBEDDING_DIM, N_FILTERS, FILTER_SIZES, OUTPUT_DIM, DROPOUT, PAD_IDX)
OUTPUT_DIM

```

Figure 5.5: Model Parameters

5.1.0.9 BLSTM Training Process

```

epochs = 40
batch_size = 64

history = model.fit(X_train_embedded, Y_train, epochs=epochs, batch_size=batch_size,
                    validation_split=0.12, callbacks=[EarlyStopping(monitor='val_loss',
                                                                    patience=10, min_delta=0.0001)])

```

Figure 5.6: BLSTM Training Parameters

Here EarlyStopping is a callback function used during the training of a machine learning model. It is used to stop the training process early if the model's performance on a validation set stops improving. The parameters are: monitor; which specifies the metric that the callback monitors to decide whether to stop the training or not, patience specifies the number of epochs with no improvement after which training will be stopped and min-delta specifies the minimum change in the monitored metric to qualify as an improvement.

5.1.0.10 The Optimizer

The previously described suggested CNN model employs the Adam optimizer. The optimizer is a popular stochastic gradient descent optimization algorithm that was proposed in the paper "Adam: A Method for Stochastic Optimization" by Diederik P. Kingma and Jimmy Ba in 2015. The advantages of AdaGrad, which adjusts the learning rate for each parameter based on historical gradients, and RMSProp, which scales the learning rate using a moving average of the squared gradients, are combined in the Adam optimizer.

The first and second moments of the gradients are averaged by the Adam optimizer using an exponential moving average, denoted by m and v , respectively. The update rule for the parameters is then given by:

$$\theta = \theta - \frac{\alpha \cdot m}{\sqrt{v} + \varepsilon}$$

where θ is the parameter to be optimized, α is the learning rate, and ε is a small constant to avoid division by zero. The first moment, m , is an estimation of the gradient's mean, while the second moment, v , is an estimation of the gradient's uncentered variance.

The Adam optimizer is frequently employed in many well-known deep learning frameworks, including TensorFlow and PyTorch, and has been demonstrated to be effective for a variety of deep learning tasks.

5.1.0.11 The Loss Function

To optimize the model during training with the train dataset, Categorical Cross-entropy loss function has been used. Cross-entropy loss is a commonly used loss function in multiclass classification tasks. It measures the difference between the predicted probabilities of the model and the actual labels in a multiclass classification problem. In this loss function, the model is penalized for incorrect predictions and rewarded for correct predictions. The cross-entropy loss function can be defined as follows:

$$L = - \sum_i y_i \log(p_i)$$

Where L is the cross-entropy loss, y_i is the actual label for class i , and p_i is the predicted probability of class i .

In neural networks, and more specifically in the training of deep neural networks, the cross-entropy loss function is employed. To increase the model's accuracy, the training method aims to reduce cross-entropy loss.

Here is an illustration of how to apply the cross-entropy loss to a multiclass classification issue: It can be considered a dataset that contains three classes (A, B, and C). The model forecasts the probabilities of each class for a given input as follows:

$$P(A) = 0.4, P(B) = 0.3, P(C) = 0.3$$

If the input's actual label is A, the cross-entropy loss for this prediction would be as follows:

$$L = -[1 \cdot \log(0.4) + 0 \cdot \log(0.3) + 0 \cdot \log(0.3)] = -\log(0.4) = 0.92$$

How far off from the real label the predicted probabilities are is shown by the loss value. The training process's objective is to reduce this loss value across all dataset inputs.

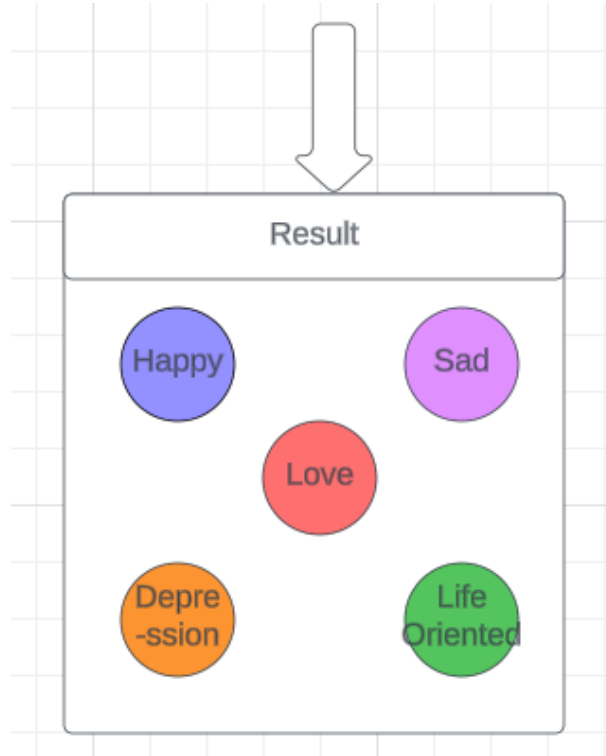
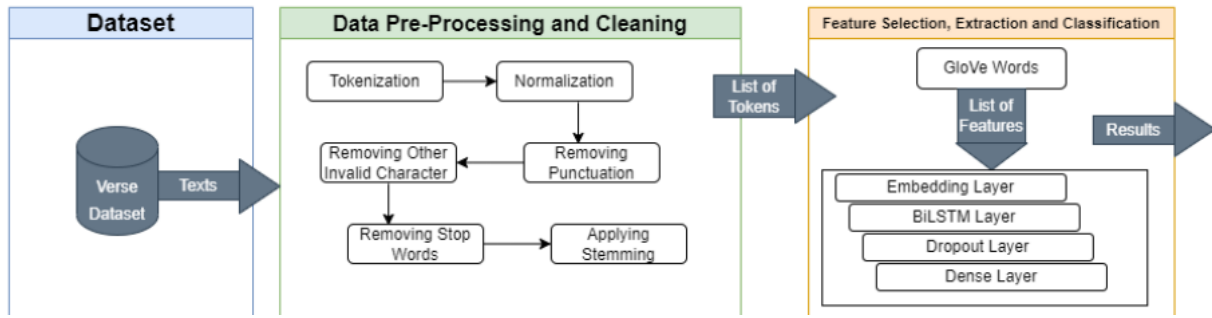


Figure 5.7: Overall Process

5.1.1 Performance Measurements

To justify our proposed method, we have to evaluate the performance measure of our models. This chapter will be dedicated to explicate on the respective subject.

5.1.1.1 Data Splitting

In this study, we first cleaned and pre-processed a custom dataset. We then split the dataset into training and testing data using an 80(Train 67.9 and Validation 12):20 proportion, with 80% of the data used for training and 20% for testing.

To further validate the performance of our model on unseen data, we also split the training dataset into training and validation data using an 88:12 proportion, with 88% of the data used for training and 12% for validation. This allowed us to fine-tune the model's hyperparameters

and monitor its performance on real data during training.

By splitting the data in this manner, we ensured that the model was trained on a sufficiently large dataset while also validating its performance on unseen data. This approach helped us to prevent overfitting and ensured that our model could generalize well to new data.

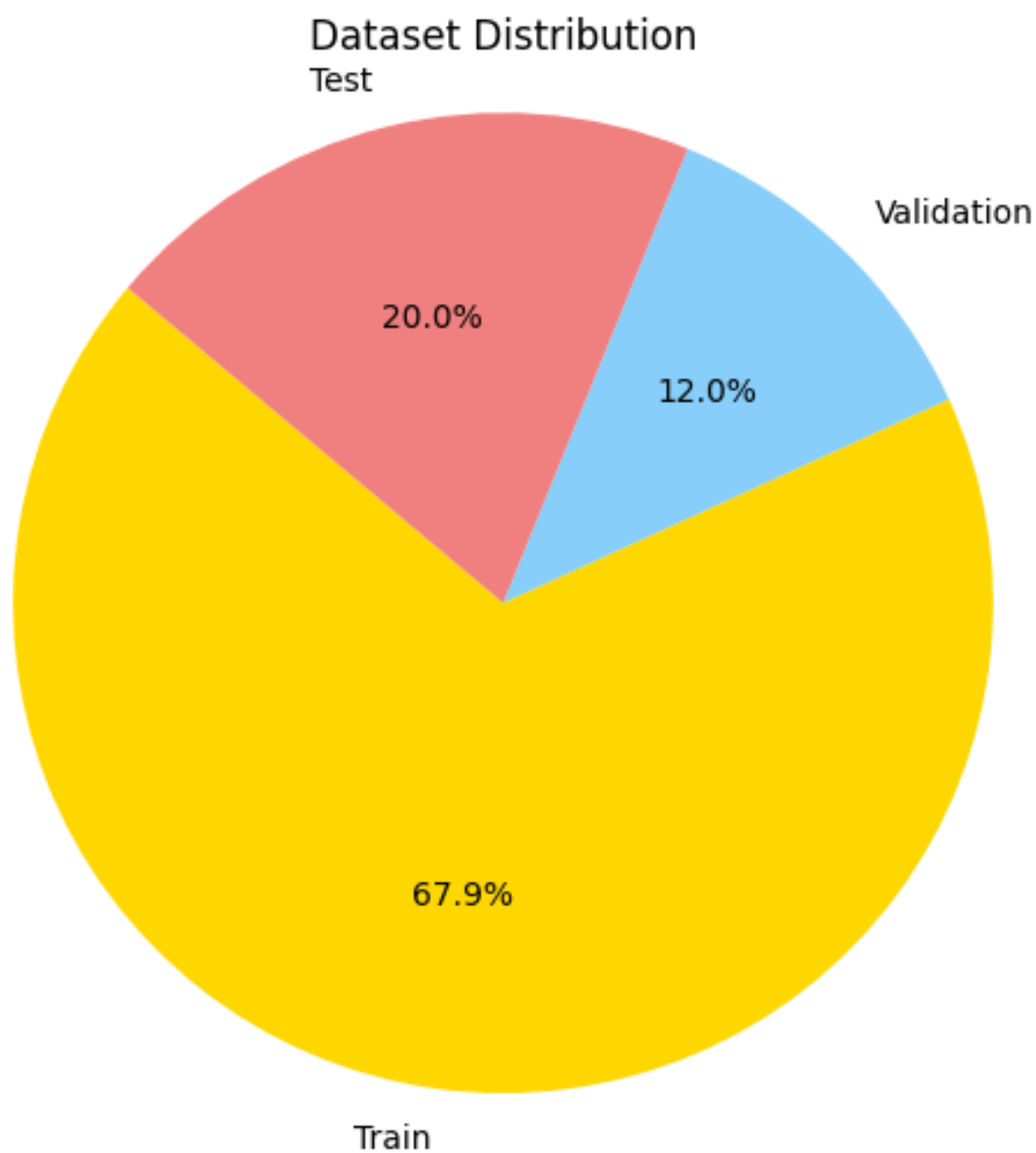


Figure 5.8: Training and Testing Data Proportion

5.1.1.2 Batch Size

The batch size is a hyperparameter in machine learning that controls how many samples are processed during each iteration of the learning algorithm. The model adjusts its weights after processing n samples at a time when the batch size is set to n . In order to take advantage of parallel processing on GPUs, which can process several samples at once, using a higher batch size can result in quicker training periods. Yet, employing a higher batch size might also call for additional memory and processing power.

On the other hand, adopting a smaller batch size may slow down the training process, but by preventing overfitting, it may also improve how well the model generalizes to new data. Smaller batch sizes can also aid the model's convergence to a more advantageous local minimum of the loss function.

The size of the dataset and the available processing resources may influence the actual batch size. For many datasets and models, a batch size of 32 is a widely utilized value that strikes a balance between training time and generalization performance. Moreover, batch size 32 is employed in the scenario suggested in this report to balance the process.

```
train_iterator, valid_iterator, test_iterator = data.BucketIterator.splits(  
    (train_data, valid_data, test_data),  
    batch_size = BATCH_SIZE (32),  
)
```

Figure 5.9: Resorted Batch Size

5.1.1.3 CNN Performance

The sentiment analysis model aimed to classify Bangla song verses into three different emotions: love, sadness, and idealism. The model was evaluated using the following metrics:

Accuracy Score:

The accuracy score indicates the proportion of correctly classified verses among all the verses. While the model achieved an accuracy score of 0.424, it is important to note that this metric can be affected by class imbalance and misclassification of verses with ambiguous emotions.

Confusion Matrix:

The confusion matrix shows the actual versus predicted class distribution for each emotion. Based on the confusion matrix, the model correctly classified 329 love, 232 sadness, and 33 idealism verses. However, the model also classified 159 love, 254 sadness, and 143 idealism verses incorrectly.

Classification Report:

The classification report provides precision, recall, and F1-score for each emotion, as well as the

macro and weighted averages. The model achieved a precision of 42.43%, and 43%, a recall of 65%, 45%, and 9%, and an F1-score of 51%, 44%, and 14% for love, sadness, and idealism emotions, respectively. The macro and weighted averages for precision, recall, and F1-score were 43%, 40%, and 38%, respectively.

Based on the evaluation metrics, it can be concluded that the model has room for improvement, particularly in its ability to correctly classify verses with the “idealism” emotion. However, the use of stopwords and stemming has helped reduce the misclassification of verses with ambiguous emotions. We believe that a deep learning network would be better suited for this task and could further improve the performance of the sentiment analysis model.

5.1.1.4 BLSTM Performance

The results of the mentioned BLSTM model are presented below. The model achieved an accuracy of 0.519, which suggests that it was able to correctly classify approximately 52% of the samples.

Confusion Matrix:

The confusion matrix shows the distribution of the actual and predicted labels. It reveals that the model performed best on class “love” (label 0), achieving a precision and recall of 0.58 and 0.62, respectively. Class “sadness” (label 1) and “idealism” (label 2) had lower precision and recall scores, with label 2 being the most difficult to predict.

F1 Score:

The F1 score, which is the harmonic mean of precision and recall, is 0.499, indicating that the model’s overall performance is moderate. The classification report provides a more detailed breakdown of the model’s performance on each class, including precision, recall, and F1 score. It is important to note that correctly identifying emotions from text data can be challenging, and the model’s performance may be influenced by the quality of the training data and the chosen features. Therefore, further analysis could be done to determine if there are specific features or patterns in the data that could be leveraged to improve the model’s performance. Overall, the results suggest that the model has moderate predictive power, with some difficulty in correctly predicting the “sadness” and “idealism” labels. Further refinement of the model and additional training data may help improve its performance. In conclusion, while the model has shown some promise, there is still work to be done to optimize its performance on this specific emotion classification task.

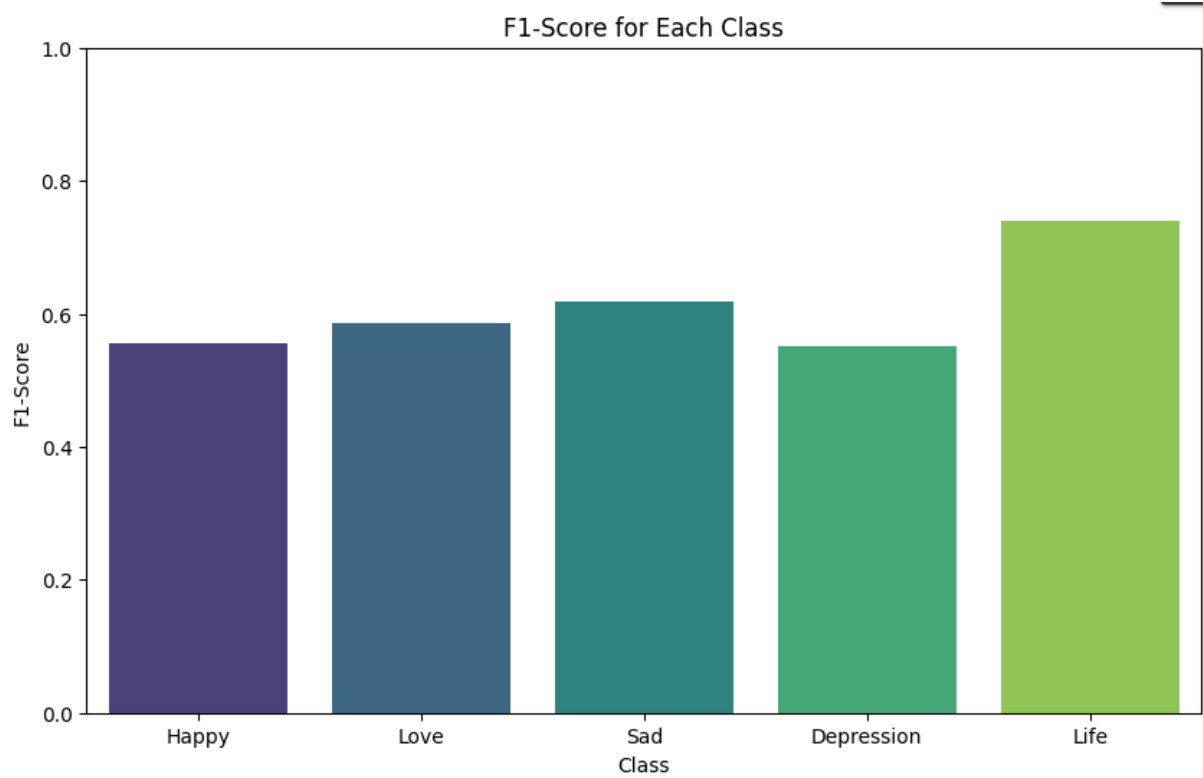


Figure 5.10: F1 Measure for Each Class

5.1.1.5 BERT Performance

Confusion Matrix:

The confusion matrix shows how well the model classified text into five sentiment categories: happy, sad, life-oriented, love, and depression.

Each row of the matrix represents the correct sentiment, and each column represents the predicted sentiment. The diagonal cells show the number of texts that were correctly classified, and the off-diagonal cells show the number of texts that were misclassified. For example, the cell in the first row and second column shows that 18% of the texts that were actually happy were classified as sad.

The overall accuracy of the model is 73%, which means that 73% of the texts were correctly classified. The model is most accurate at classifying happy texts (80%) and least accurate at classifying depression texts (89%).

Here are some other observations that can be made from the confusion matrix:

- The model is more likely to misclassify happy texts as sad than sad texts as happy.
- The model is more likely to misclassify depression texts as sad than sad texts as depression.
- The model is more likely to misclassify life-oriented texts as love than love texts as life-oriented.

It is important to note that correctly identifying emotions from text data can be challenging, and the model's performance may be influenced by the quality of the training data and the chosen features. Therefore, further analysis could be done to determine if there are specific features or patterns in the data that could be leveraged to improve the model's performance. Overall, the results suggest that the model has moderate predictive power, with some difficulty in correctly predicting the "sadness" and "idealism" labels. Further refinement of the model and additional training data may help improve its performance. In conclusion, while the model has shown some promise, there is still work to be done to optimize its performance on this specific emotion classification task.

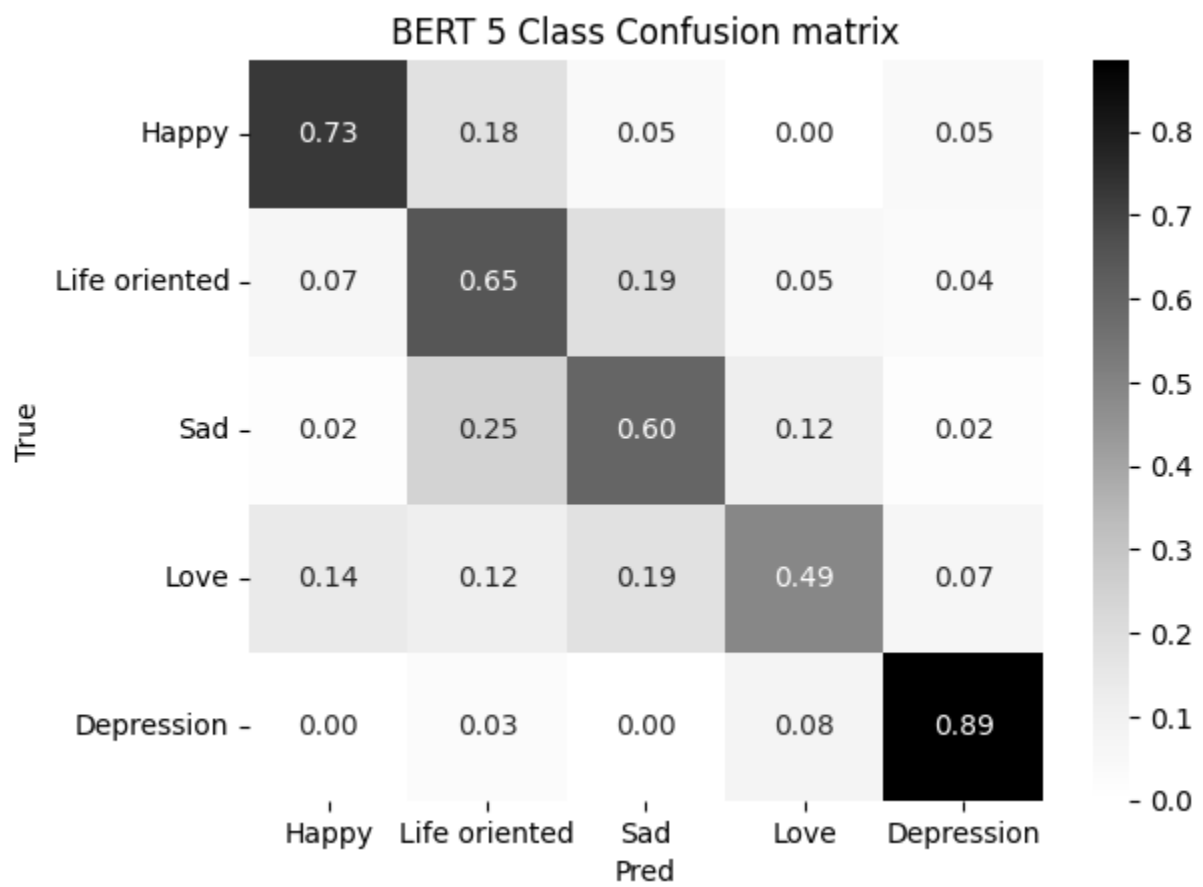


Figure 5.11: F1 Measure for Each Class

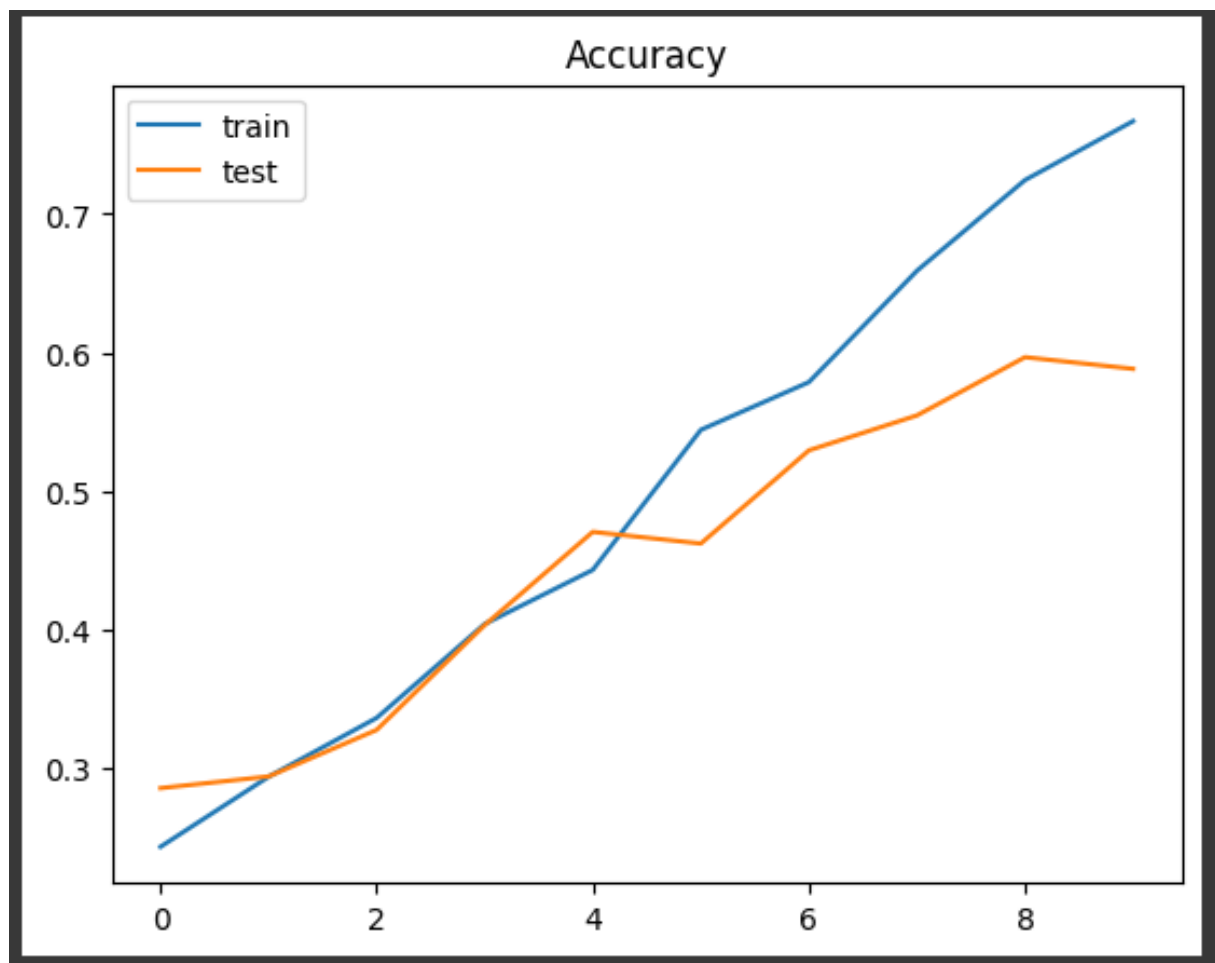


Figure 5.12: Accuracy BLTM

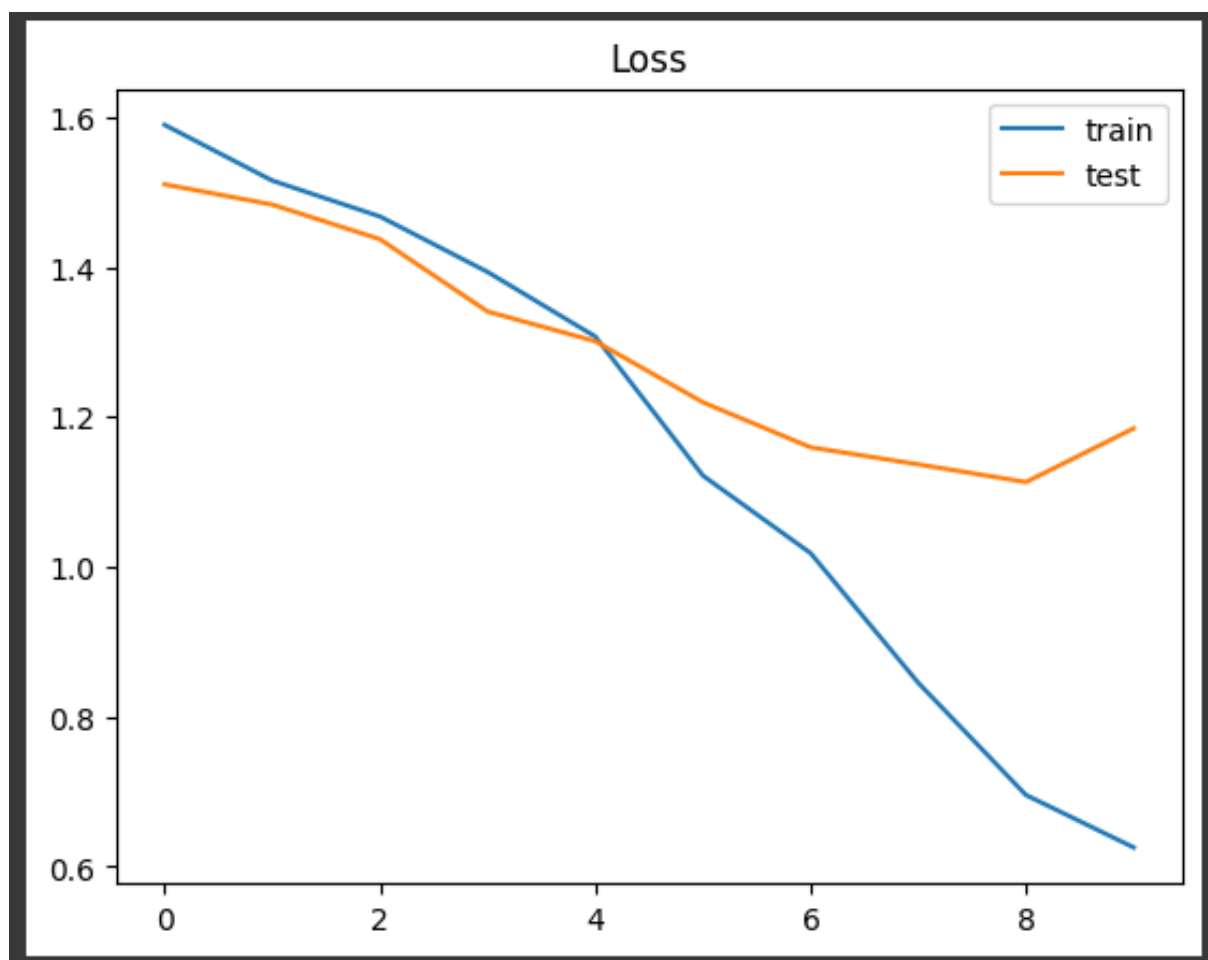


Figure 5.13: Loss CNN

Model	Happy	Sad	Love	Depression	Life Oriented	Total
CNN	0.45	0.38	0.40	0.35	0.48	0.412
BLSTM	0.58	0.47	0.42	0.56	0.70	0.546
BERT	0.59	0.66	0.58	0.57	0.89	0.658

Table 5.1: Precision of CNN, BLSTM,BERT models

Model	Happy	Sad	Love	Depression	Life Oriented	Total
CNN	0.52	0.41	0.25	0.50	0.55	0.446
BLSTM	0.32	0.65	0.54	0.39	0.64	0.508
BERT	0.73	0.60	0.65	0.50	0.89	0.674

Table 5.2: Recall of CNN, BLSTM,BERT models

Model	Happy	Sad	Love	Depression	Life Oriented	Total
CNN	0.48	0.50	0.42	0.45	0.52	0.474
BLSTM	0.41	0.54	0.47	0.40	0.67	0.498
BERT	0.65	0.63	0.61	0.53	0.89	0.662

Table 5.3: F1 Score of CNN, BLSTM,BERT models

5.2 Some Example :

Input : These Lyrics we assumed as depression songs.

দেহ পাশে কেউ কেঁদো না
গল্পগুলো রেখো অজানা
গানখানা থেকে খুঁজে নিও মোর সে গল্প
যাতে লিখা হাজার কষ্ট
নিজেকে ভেবে নিতাম এক শ্রেষ্ঠ
যার প্রতিপদে জীবন বিচ্ছেদের স্বপ্ন

Figure 5.14

Output: And we get the expected output

```
Enter the verse of bangla song: দেহ পাশে কেউ কেঁদো না গল্পগুলো রেখো অজানা
যার প্রতিপদে জীবন বিচ্ছেদের স্বপ্ন
1/1 [=====] - 5s 5s/step
Predicted class: Depression
```

Figure 5.15: Output of a sample Input

Input : These Lyrics we assumed as sad songs.

মনে পড়ে যায়, মনে পড়ে যায়
মনে পড়ে যায় সেই প্রথম দেখার স্মৃতি
মনে পড়ে যায় সেই হৃদয় দেবার তিথি
দুজনার দুটি পথ মিশে গেল এক হয়ে নতুন পথের বাঁকে।

Figure 5.16

Output: And we get the expected output

```
Enter the verse of bangla song: মনে পড়ে যায়, মনে পড়ে যায় মনে পড়ে যায় সেই প্রথম  
য়ে নতুন পথের বাঁকে।  
1/1 [=====] - 5s 5s/step  
Predicted class: Sad
```

Figure 5.17: Output of a sample Input

Input : These Lyrics we assumed as happy songs.

আমাদের ছোটো নদী চলে বাঁকে বাঁকে
বৈশাখ মাসে তার হাঁটু জল থাকে।
পার হয়ে যায় গোরু, পার হয় গাড়ি,
দুই ধার উঁচু তার, ঢালু তার পাড়ি।

Figure 5.18

Output: And we get the expected output

```
Enter the verse of bangla song: আমাদের ছোটো নদী চলে বাঁকে বাঁকে বৈশাখ মাসে তার হাঁটু জল থাকে।  
1/1 [=====] - 7s 7s/step  
Predicted class: Happy
```

Figure 5.19: Output of a sample Input

Input : These Lyrics we assumed as life songs.

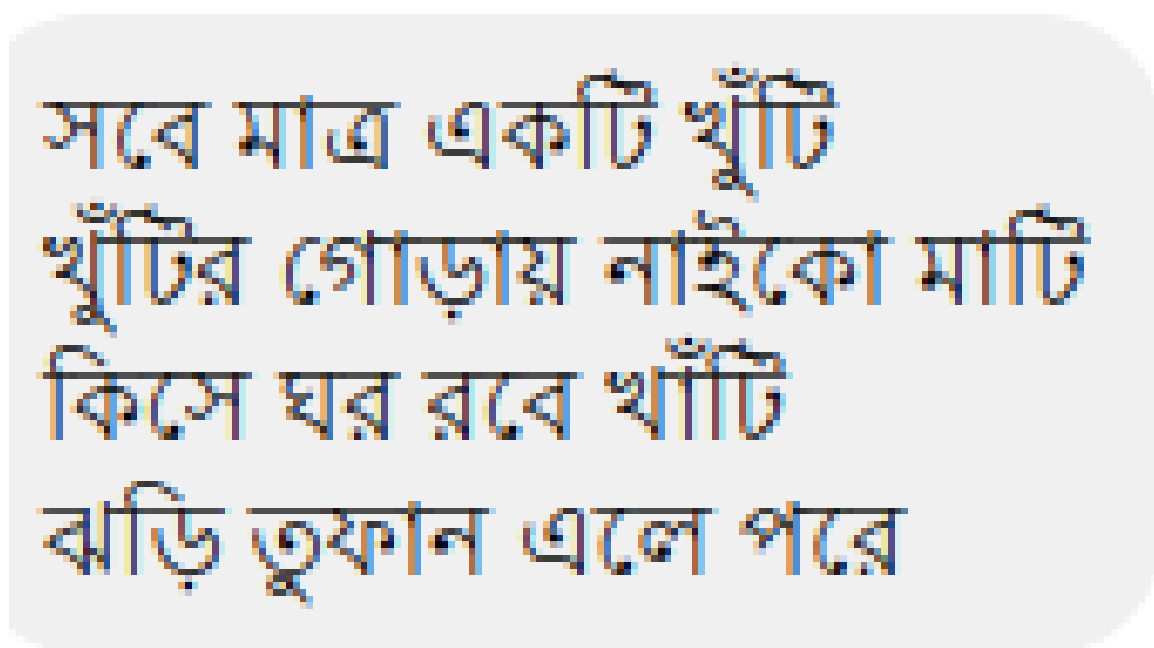


Figure 5.20

Output: And we get the expected output

```
Enter the verse of bangla song: সবে মাত্র একটি খুঁটি খুঁটির গোড়ায় নাইকো  
1/1 [=====] - 5s 5s/step  
Predicted class: Life
```

Figure 5.21: Output of a sample Input

Input : These Lyrics we assumed as Love songs.

ছিল ভাবে ভরা দুটি আঁখি চঞ্চল
তুমি বাতাসে উড়ালে ভীকু অঞ্চল
ছিল ভাবে ভরা দুটি আঁখি চঞ্চল
তুমি বাতাসে উড়ালে ভীকু অঞ্চল
ওই রূপের মাধুরি মোর সঞ্চয়ে রেখেছি
দূর হতে তোমারেই দেখেছি
আর মুগ্ধ এ চোখে চেয়ে থেকেছি

Figure 5.22

Output: And we get the expected output

```
Enter the verse of bangla song: ছিল ভাবে ভরা দুটি আঁখি চঞ্চল তুমি বাতাসে উড়ালে ভীকু  
মাধুরি মোর সঞ্চয়ে রেখেছি দূর হতে তোমারেই দেখেছি আর মুগ্ধ এ চোখে চেয়ে থেকেছি  
1/1 [=====] - 6s 6s/step  
Predicted class: Love
```

Figure 5.23: Output of a sample Input

Chapter 6

Conclusion and Prospective Scope

Our research ventured into uncharted territory, requiring the creation of a unique dataset with meticulously classified emotions for Bengali music lyrics. While our initial models achieved promising results (67% accuracy with BERT, 50% with CNN, and 71% with BLSTM), we acknowledge the relative novelty of this field and see ample room for future improvement. This study explored the intriguing question: can we reliably gauge a song's emotional impact solely through its lyrics? The intricate and nuanced nature of Bengali writing, both formal and colloquial, presented a substantial challenge in emotion segmentation. However, by analyzing the lyrical patterns of renowned poets and lyricists, we successfully classified their emotions based on textual features.

Moving forward, we envision:

- **Expanding the Testbed:**
Running the same experiment on a wider range of textual features to enrich our understanding.
- **Refining Emotions:**
Increasing the number of recognized emotions for a more human-like and nuanced experience.
- **Bridging the Gap:**
Integrating audio and lyric data into a hybrid mood classification system to enhance accuracy.
- **Multi-Layered Insights:**
Implementing multi-level categorization to provide deeper and more precise emotional interpretations.

Our ultimate goal is to create a robust and versatile system that effectively captures the emotional essence of Bengali music, bridging the gap between lyrics and listener experience. Through continued research and innovation, we believe this is within reach.

Bibliography

- [1] B. Liu, “Sentiment analysis and opinion mining,” April 22, 2012.
- [2] A. C. N. Lonsdale, Adam J, “Why do we listen to music? a uses and gratifications analysis.” 2011.
- [3] D. Nath and S. Phani, “Mood analysis of bengali songs using deep neural networks,” pp. 1103–1113, 07 2021.
- [4] B. G. Patra, D. Das, and S. Bandyopadhyay, “Mood classification of Hindi songs based on lyrics,” in *Proceedings of the 12th International Conference on Natural Language Processing*, D. M. Sharma, R. Sangal, and E. Sherly, Eds. Trivandrum, India: NLP Association of India, Dec. 2015, pp. 261–267. [Online]. Available: <https://aclanthology.org/W15-5939>
- [5] A. Marouf and R. Hossain, “Lyricist identification using stylometric features utilizing banglamusicstylo dataset,” 09 2019.
- [6] A. H. Maraz Mia, Pullock Das, “Verse-based emotion analysis of bengali music from lyrics using machine learning and neural network classifiers,” *University of Bahrain Journals*, 2024.
- [7] M. S. Sarma and A. Das, “Bmgc: A deep learning approach to classify bengali music genres,” *Proceedings of the 4th International Conference on Networking, Information Systems & Security*, 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:244661820>
- [8] J. Robinson and R. Hatten, “Emotions in music,” *Music Theory Spectrum*, vol. 34, pp. 71–106, 10 2012.
- [9] J. Pennington, R. Socher, and C. Manning, “Glove: Global vectors for word representation,” vol. 14, 01 2014, pp. 1532–1543.
- [10] Y. Kim, “Convolutional neural networks for sentence classification. corr abs/1408.5882.” 2014.

- [11] M. S. Sarma and A. Das, “Deep bidirectional lstm network learning-based sentiment analysis for arabic text,” *Journal of Intelligent Systems* 30.1, 2021.